

파머완 4장(분류) -Part1

4장. 분류

01. 분류의 개요

분류

- 지도학습의 대표적인 유형
- 학습 데이터로 주어진 데이터의 피쳐와 레이블값 (결정 값 , 클래스 값) 을 머신러닝 알고리즘으로 학습해 모델 생성 및 예측
- 분류는 다양한 머신러닝 알고리즘으로 구현 가능
 - 나이브 베이즈
 - 로지스틱 회귀
 - 결정 트리
 - 서포트 벡터 머신(SVM)
 - 최소 근접 알고리즘

- 신경망
- 앙상블

앙상블

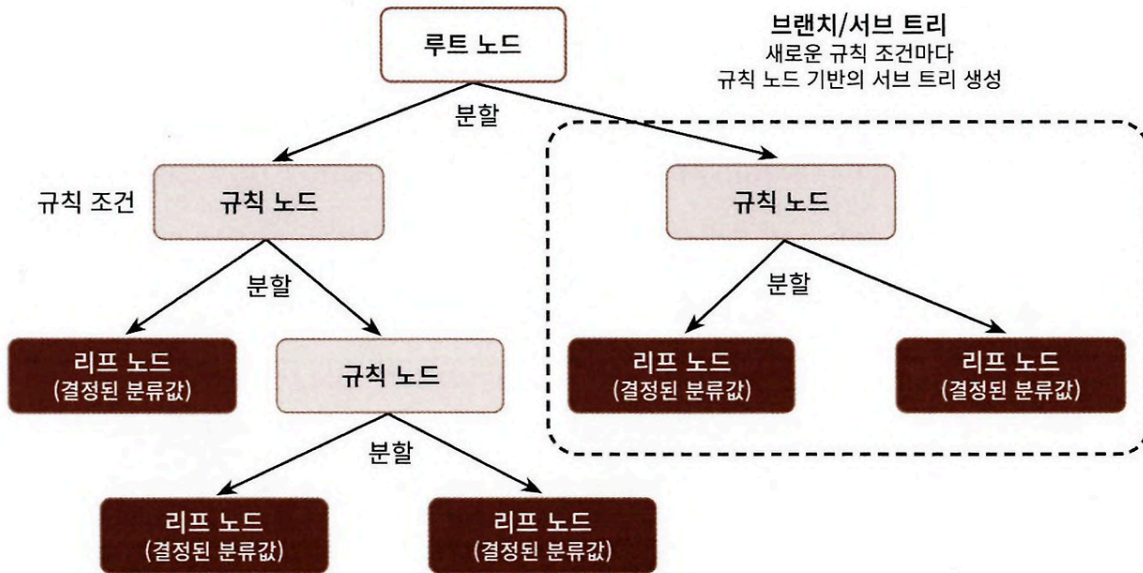
- 분류에서 가장 각광 받는 방법 중 하나.
- 보팅(Voting), 배깅(Bagging), 부스팅(Boosting), 스택킹(Stacking) 방식 등이 있음
 - 보팅 : 서로 다른 알고리즘을 가진 분류기가 투표를 통해 최종 예측 결과 결정.
 - 배깅 : 같은 유형의 알고리즘, 데이터 샘플은 다르게 한 분류기들이 투표를 통해 최종 예측 결과 결정 - 랜덤포레스트
 - 부스팅 : 모델을 순차적으로 학습하여 이전 모델 보완 - 그래디언트 부스팅, XgBoost, LightGBM
 - 스택킹 : 여러 모델의 예측값을 다시 학습 데이터로 만들어서, 다른 모델(메타 모델)로 재학습시켜 결과 예측.

결정트리 - 앙상블을 구성하는 하나의 기본 학습기

- 앙상블의 기본 알고리즘
- 단점 존재 : 예측 성능을 향상시키기 위해 복잡한 규칙 구조를 가져야 하며 , 과적합이 발생해 예측 성능 저하.
 - → 앙상블에서 이러한 약한 학습기를 결합해 확률적 보완 & 가중치 업데이트로 예측 성능 향상시킴.

02. 결정 트리

- 데이터를 if/else 조건에 따라 반복적으로 분할. if/else 기준을 자동으로 찾아내 예측을 위한 규칙을 만드는 알고리즘.
- 구조
 - 규칙 노드: 규칙 조건이 됨. 리프 노드: 결정된 클래스값.
 - 서브 트리: 새로운 규칙마다 생성 됨.



- 트리의 깊이(depth)가 깊어질수록, 과적합으로 이어지기 쉬워 결정 트리의 예측 성능이 저하될 가능성이 높아짐.
- **정보 균일도가 높은 데이터 세트를 먼저 선택할 수 있도록 규칙 조건을 만듦.** 즉, 결정 트리는 데이터를 나눌 때, 한쪽으로 나눈 결과가 최대한 한 종류로 모이도록(정보가 같도록, 균일하도록) 만드는 조건을 우선 선택한다.
 - ex. 노랑색블록은 모두 동그라미, 빨강과 파랑블록은 동그라미, 네모, 세모 -> 규칙 조건 'if 색깔==노란색'.
 - 노란색 블록이면 모두 동그라미 블록으로 가장 쉽게 예측 가능. 나머지 빨강, 파랑 블록에 대해서는 다시 균일도 조건 찾기.

정보의 균일도 측정 방식

1. 정보 이득 지수

- 1- 엔트로피
 - cf. 엔트로피: 주어진 데이터 집합의 혼잡도. 서로 다른 값이 섞여 있으면 엔트로피가 높다.
- 결정트리에서는 **정보 이득이 높은 속성을 기준으로 분할**

2. 지니 계수

- 사이킷런의 DecisionTreeClassifier에서 사용.

- 0이 가장 평등, 1로 갈수록 불평등.
- 지니계수가 낮다 = 데이터 균일도가 높다 → 결정트리에서는 지니 계수가 낮은 속성을 기준으로 분할

결정 트리 모델의 특징

장점

- '균일도' 기반 알고리즘이 쉽고 직관적
- 피처 스케일링과 정규화 같은 전처리 과정 필요x

단점

- 결정 트리는 정확도를 높이기 위해 조건을 계속 추가하면서 트리 깊이가 커짐 → 과적합으로 성능↓
- 트리의 크기를 사전에 제한하는 튜닝 필요

결정 트리 파라미터

- 결정 트리 알고리즘을 위한 DecisionTreeClassifier(분류), DecisionTreeRegressor(회귀) 클래스 제공.
- 두 클래스는 동일한 파라미터 사용

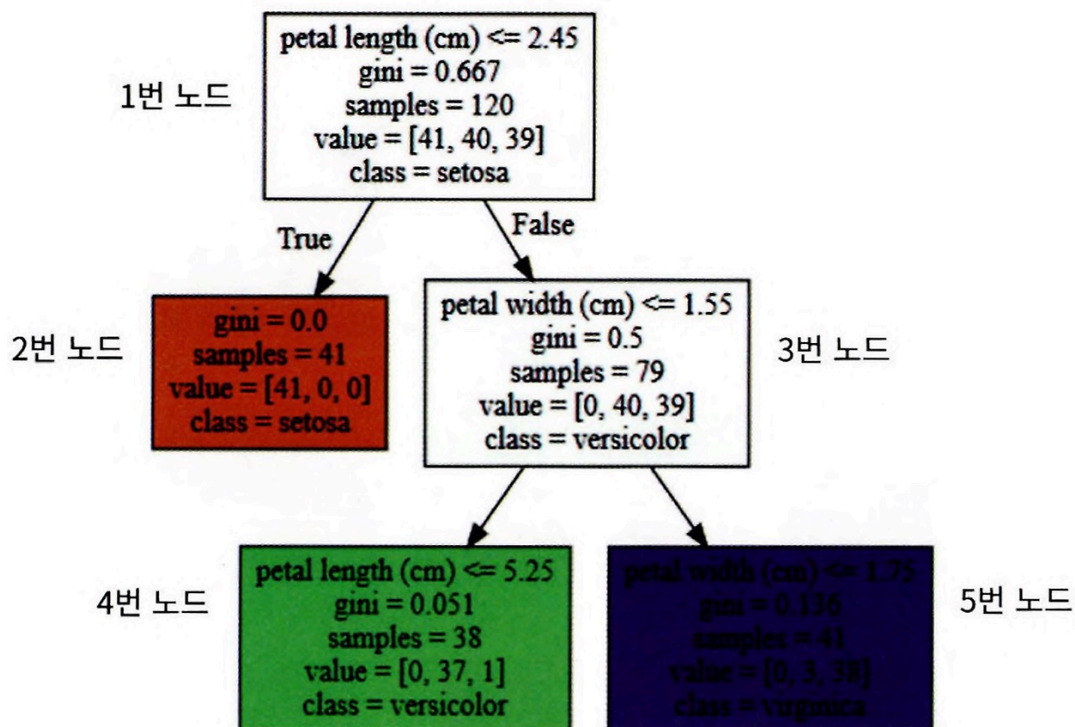
파라미터명	설명
<code>min_samples_split</code>	노드를 분할할 수 있는 최소한의 샘플 데이터 수. → 클수록 과적합 제어 디폴트: 2
<code>min_samples_leaf</code>	분할이 될 경우 왼, 오 브랜치 노드에서 가져야 할 최소한의 샘플 데이터 수 큰값일수록, 분할 덜함. 과적합 제어 (비대칭적 데이터) 작게 설정 필요.
<code>max_features</code>	최적의 분할을 위해 고려할 최대 피처수. 디폴트: None(모든 피처) int형 지정: 대상피처개수 / float형 지정 : 전체 피처 중 대상 피처의 퍼센트 sqrt, auto / log 등..

max_depth	트리의 최대 깊이 규정. 디폴트: None(최대한 분할) → 적절한 값으로 제어 필요.
max_leaf_nodes	말단 노드의 최대 개수

결정 트리 모델의 시각화

cf. `export_graphviz()` : Graphviz 가 읽어들이어서 그래프 형태로 시각화할 수 있는 출력 파일 생성.

- 사이킷런의 트리 모듈은 Graphviz를 사용하기 위해 `export_graphviz()` 함수 제공. **from sklearn.tree import export_graphviz**
→ 학습된 결정트리가 어떤 구조와 규칙으로 만들어졌는지 그림으로 볼 수 있다.
- 인자: 학습이 완료된 estimator, out_file= output 파일명, class_names=결정클래스의 명칭, feature_names=피처의 명칭



노드 지표

- 첫번째 째 줄 조건: 자식 노드를 만들기 위한 규칙 조건. 이 조건이 없으면 리프노드

- **gini** : value로 주어진 데이터 분포에서의 지니계수
- **samples**: 현 규칙에 해당하는 현재 데이터 건수
- **value=[]**: 클래스 값 기반의 데이터 건수.
- **class** : value에서 가장 큰 값을 가지는 클래스명, 해당 노드에서 예측하는 클래스.(= 이 노드에 새로운 데이터가 들어왔을 때, 결정트리가 어느 클래스에 속한다고 판단하는 지)
- cf. 각 노드의 색깔: 데이터의 레이블 값. 색깔이 진할수록 지니계수가 낮고, 해당 레이블에 속하는 샘플 데이터↑
 - ex. 주황색:0(Setosa), 초록색:1(Versicolor), 보라색: 2(Virginica)

DecisionTreeClassifier 객체의 feature_importances_ 속성

- 객체.feature_importances_
- 피처의 중요한 역할 지표를 보여줌. 즉, 피처가 트리 분할시 정보 이득이나 지니 계수를 얼마나 효율적으로 잘 개선시켰는지를 정규화된 값으로 표현.
- **값이 높다 = 해당 피처의 중요도가 높다.**
- **ndarray 형태로 반환하고, 피처 순서대로 값이 할당**
- 결정트리에서는 중요한 몇 개의 피처가 명확한 규칙 트리를 만드는 데 기여하기 때문에 중요.

결정 트리 과적합(Overfitting)

(실습 위주)

- 결정 트리의 하이퍼 파라미터를 디폴트로 하는 경우보다, **min_samples_leaf, min_samples_split, max_depth** 등을 지정하고 분류를 수행하면, 좀 더 일반화된 분류 규칙에 따라 분류 가능.
- 학습 데이터에만 지나치게 최적화된 분류 기준은 오히려 테스트 데이터 세트에서 정확도를 떨어뜨릴 수 있음.

결정 트리 실습 - 사용자 행동 인식 데이터 세트

(실습 위주)

- DecisionTreeClassifier의 하이퍼 파라미터: max_depth, min_samples_split, min_samples_leaf, max_features 등 → 리스트로 하이퍼 파라미터 지정

- 결정트리 객체 생성
- GridSearchCV 로 하이퍼 파라미터 튜닝(fit)
- **bestestimator**로 (= 최적의 하이퍼 파라미터로 학습이 완료된 결정트리 estimator) 로 예측/평가(predict, accuracy)
- bestestimator.feature_importances_ 로 해당 estimator에서의 피쳐 중요도 확인

```
#max_depth, min_samples_split 같이 변경하며 확인
params = {
    'max_depth' : [8, 12, 16, 20],
    'min_samples_split' : [16, 24 ],
}

grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring='accuracy',
cv=5, verbose=1)
grid_cv.fit(x_train, y_train)
print('GridSearchCV 최고 평균 정확도 수치:
{0:.4f}'.format(grid_cv.best_score_))
print('GridSearchCV 최적 하이퍼 파라미터:', grid_cv.best_params_)

#최고의 하이퍼 파라미터 조합으로 학습이 완료된 Estimator를 가지고 예측 수행.
best_df_clf = grid_cv.best_estimator_
pred1 = best_df_clf.predict(X_test)
accuracy = accuracy_score(y_test, pred1)
print('결정 트리 예측 정확도:{0:.4f}'.format(accuracy))

#피쳐 중요도
ftr_importances_values = best_df_clf.feature_importances_
ftr_importances = pd.Series(ftr_importances_values,
index=x_train.columns)
ftr_top20=ftr_importances.sort_values(ascending=False)[:20]
```

- 각 피쳐의 중요도를 feature_importances 속성으로 알아보기 :

```
ftr_importances_values = best_df_clf.feature_importances_ #1차원
ndarray
ftr_importances = pd.Series(ftr_importances_values,
index=x_train.columns) #시리즈로 만들기
```

```
ftr_top20=ftr_importances.sort_values(ascending=False)[:20] #시리즈의 정렬 후, 상위 20개
```

03. 앙상블 학습

앙상블 학습 개요

- 앙상블 학습을 통한 분류 : 여러 개의 분류기(Classifier)를 생성하고 그 예측을 결합함으로써 보다 정확한 최종 예측을 도출하는 기법
- 대부분의 정형 데이터 분류 시, 앙상블이 뛰어난 성능을 보이고 애용됨.
- 목표: 단일 분류기보다 신뢰성이 높은 예측값 얻기
- 학습 유형: 보팅(Voting), 배깅(Bagging), 부스팅(Boosting) / 스택킹(Stacking)

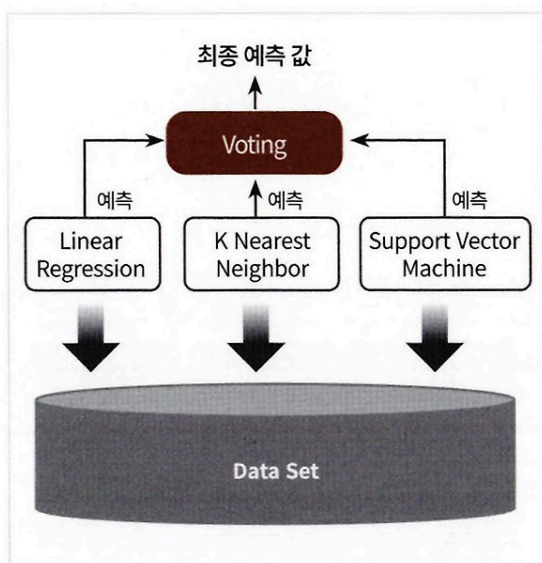
보팅과 배깅

공통점

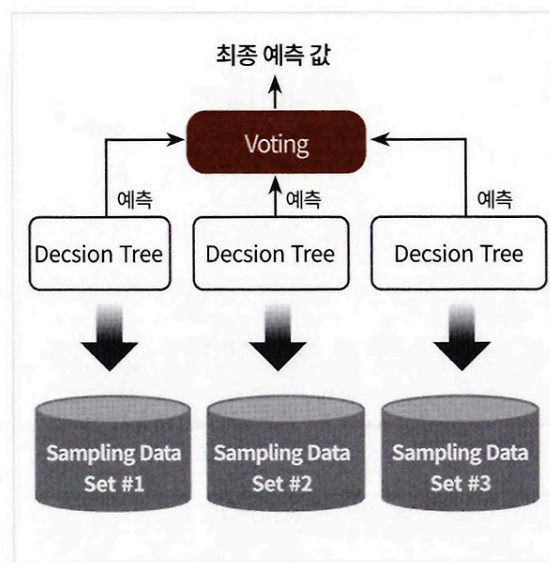
- 여러 개의 분류기가 투표를 통해 최종 예측 결과 결정.

차이점

- 보팅: 서로 다른 알고리즘을 가진 분류기의 결합
- 배깅: 각각의 분류기가 같은 알고리즘 기반이지만, 데이터 샘플링을 서로 다르게 함.



Voting 방식



Bagging 방식

보팅 유형 - 하드 보팅(Hard Voting)과 소프트 보팅(Soft Voting)

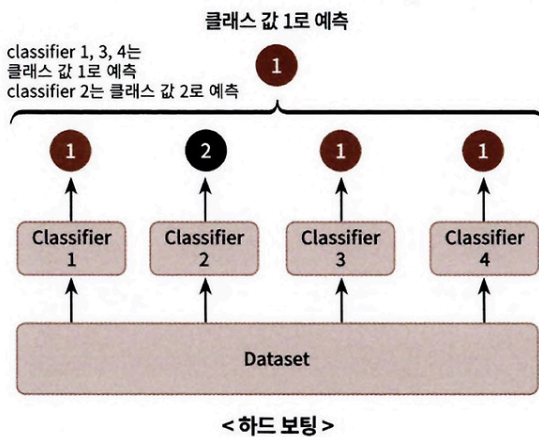
하드 보팅

- 다수결 방식. 예측한 결과값들 중 다수의 분류기가 결정한 예측값(클래스)을 최종 보팅 결과값으로 선정
- 확률 출력 없이도 가능.

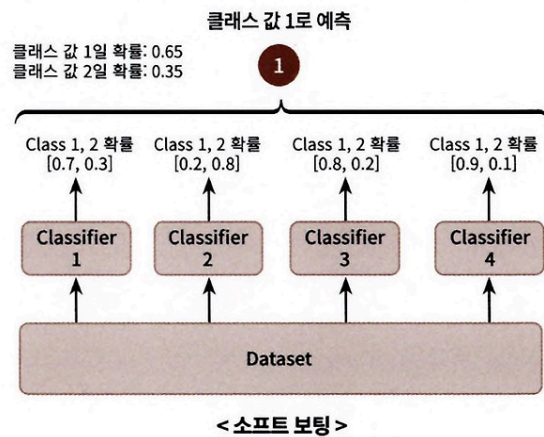
소프트 보팅

- 분류기들의 레이블 값 결정 확률을 평균하여, 이들 중 확률이 가장 높은 레이블 값을 최종 결과값으로 선정
- 클래스, 예측확률 모두 사용
- 일반적으로 하드보팅보다 예측 성능이 더 좋아 많이 사용됨.

Hard Voting은 다수의 classifier 간 다수결로 최종 class 결정



Soft Voting은 다수의 classifier들의 class 확률을 평균하여 결정



VotingClassifier 클래스 - 보팅 분류기(Voting Classifier) 만들기

- 서로 다른 분류기 모델 보팅으로 결합.
- `from sklearn.ensemble import VotingClassifier`
- 학습 - 객체.fit(X_train, y_train)
- 예측 - 객체.predict(X_test)
- 평가

파라미터

- **estimators**: 튜플 여러개를 리스트로 묶어 입력. [(‘모델이름’, 객체이름), (‘모델이름’, 객체이름), ..]

- **voting** : 'hard'이면 하드 보팅, 'soft'이면 소프트 보팅. (디폴트: 'hard')

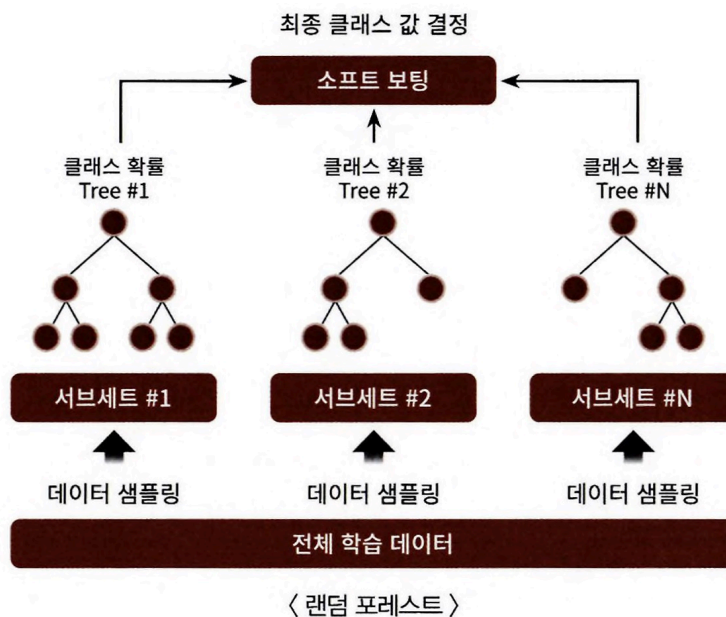
앙상블 방법의 특징

- 보팅으로 여러 개의 분류기를 결합한다고 해서, 무조건 예측 성능이 높아지지는 않는다. 하지만 대부분 단일 ML 알고리즘보다 뛰어난 성능을 가지는 경우가 많다.
- 보팅과 스택킹 등은 서로 다른 알고리즘 기반이지만, **배깅과 부스팅은 대부분 결정 트리 알고리즘 기반.**
 - 결정 트리는 과적합 위험 O → 앙상블 학습에서 많은 분류기를 결합해 이를 보완.

04. 랜덤 포레스트

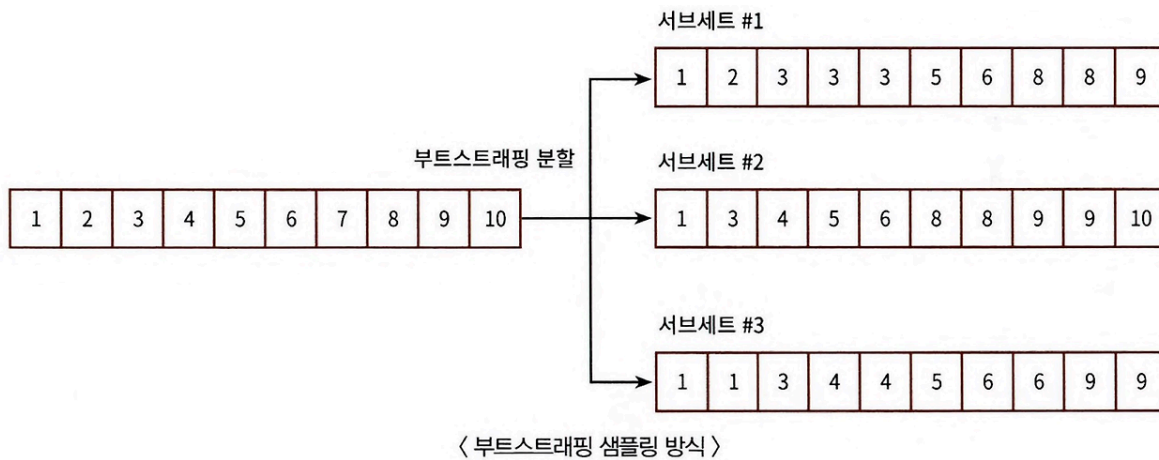
랜덤 포레스트의 개요

- 앙상블 학습의 배깅 방식의 대표적인 알고리즘, 개별 분류기는 결정트리 기반.
- 비교적 빠른 수행 속도, 높은 예측 성능
- 여러 개의 결정 트리 분류기가 전체 데이터에서 배깅 방식으로 각자의 데이터를 샘플링해 개별적 학습 → 모든 분류기가 보팅을 통해 예측 결정



부트스트래핑 분할 방식

- 여러 개의 데이터 세트를 **중첩되게** 분리하는 것
- 개별 결정트리가 학습하는 데이터 세트(서브세트 데이터) → 전체 데이터에서 중첩되게 샘플링한 데이터 세트
- 서브세트의 데이터 건수 = 전체 데이터 건수



RandomForestClassifier 클래스 - 랜덤 포레스트 기반의 분류 지원

- `from sklearn.ensemble import RandomForestClassifier`
- 학습 - 객체.fit(X_train, y_train)
- 예측 - 객체.predict(X_test)
- 평가

파라미터

n_estimators 제외한 파라미터들은 DecisionTreeClassifier 에서 나왔던 것들임. 여기서 설정하면 랜덤 포레스트 안의 모든 결정트리에 동일 적용.

- **n_estimators** : 결정트리를 몇 개 만들지. (디폴트: 10)
- **max_features** : 디폴트 auto(=sqrt) → 랜덤 포레스트의 트리를 분할하는 피처를 참조할 때 전체 피처가 아니라 sqrt(전체 피처 개수)만큼 참조.
- **min_samples_split**, **min_samples_leaf**, **max_depth** : 결정트리와 같음. 역시 과적합 개선에 사용되는 파라미터.
- **max_leaf_nodes**

랜덤 포레스트 하이퍼 파라미터 및 튜닝

(실습 위주)

- RandomForestClassifier의 하이퍼 파라미터: max_depth, min_samples_split, min_samples_leaf, max_features 등 → 리스트로 하이퍼 파라미터 지정
- 랜덤포레스트 객체 생성
- GridSearchCV 로 하이퍼 파라미터 튜닝(fit)
- **bestestimator**로 (= 최적의 하이퍼 파라미터로 학습이 완료된 랜덤포레스트 estimator)로 예측/평가(predict, accuracy)
- bestestimator.feature_importances_ 로 해당 estimator에서의 피쳐 중요도 확인